

DEPTH CHARGE

Personal Implementation Guide

Syed Alyan Hussain (25K-0648) — Member 2

Screen buffer • Flicker-free renderer • Status bar (HUD) • Explosions • Sonar & hidden mines

Your folder / files	src/alyan/ — render.inc, hud.inc, explosion.inc, sonar.inc, stubs_alyan.inc
Your working dates	Week 0 – Week 6: 5 Oct – 22 Nov 2026 (demo Mon 23 Nov)
GitHub repo	github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection
Group Leader	Muhammad Anas (25K-0899)
Member 2	Syed Alyan Hussain (25K-0648)
Member 3	Ali Raza (25K-0827)
Instructor / Demo	Muhammad Owais / Monday 23 November 2026

Read this first

- The code in this guide is a **worked example**. Type it yourself, change it if you want, and make sure you can explain **every line**. The viva is individual and covers the **whole** game, not only your part.
- Course rule: code taken from AI tools must be **disclosed** in the final report, e.g. "Reference examples from an AI-generated implementation guide were studied and adapted."
- There are three guides (one per member). They use the same files, names and dates, so they fit together.

1. Your Job in One Page

You build **everything the player sees** during the game: the screen buffer, the renderer that draws without flicker, the status bar, the explosion animation and the sonar effect that reveals hidden mines. Your part is the most *visual* one — and the others need it early to test their code, so start fast.

In simple words

- We never draw straight to the screen. We first "paint" a copy of the screen in memory (`ScreenBuf`).
- Then we compare it with what is already on the screen and only redraw the cells that changed.
- That is why the game does not flicker — and it is a great thing to explain in the viva.

1.1 Your steps

Step	Task	File (in src/alyan/)	Week · dates
—	Study, fork, clone, build, practice Pull Request	—	W0 · 5 – 11 Oct
B1	Console setup and the screen buffer	render.inc	W1 · 12 – 18 Oct
B2	Renderer and frame builder	render.inc	W2 · 19 – 25 Oct
B3	Status bar (HUD) and number printing	hud.inc	W3 · 26 Oct – 1 Nov
B4	Explosions	explosion.inc	W4 · 2 – 8 Nov
B5	Sonar mode and hidden mines	sonar.inc	W5 · 9 – 15 Nov
B6	Optional speed-up (RenderFast) and polish	render.inc	W6 · 16 – 18 Nov
—	Bug fixes only, testing, report section	docs/report/alyan.md	W6 · 19 – 22 Nov

1.2 What you give to the others / what you use from them

These are the "promises" between members. If your procedure takes and returns exactly what is written here, the three parts will fit together on the first try.

You provide	Input (registers)	Result	Used by
<code>InitSystem</code>	—	Seeds random (Ali's <code>SeedRandom</code>), hides cursor, sets title	main
<code>BuildFrame</code>	—	Draws sub, torpedoes, charges, explosions, HUD into <code>ScreenBuf</code>	GameLoop
<code>Render</code>	—	Shows only changed cells on the console	GameLoop
<code>InvalidateScreen</code>	—	Next Render redraws everything	InitGame
<code>StartExplosion</code>	AL = col, AH = row	Starts a 5-frame explosion	CheckCollisions
<code>UpdateExplosions</code>	—	Ages explosions, frees finished ones	GameLoop
<code>StartSonar</code> , <code>UpdateSonar</code>	—	Turn sonar on / end it after 2 s	HandleInput, GameLoop

You use	Owner	What it does for you
<code>subX</code> , <code>Torpedoes</code> , <code>gameState</code>	Anas	Where to draw the sub and torpedoes; current state
<code>Charges</code> , <code>score</code> , <code>lives</code> , <code>level</code>	Ali	What to draw for charges and the status bar

You use	Owner	What it does for you
<code>SeedRandom</code>	Ali	Called inside your <code>InitSystem</code>
<code>PlaySonarSweep</code>	Ali	Sound played when sonar starts

2. Your Weekly Timeline (5 Oct – 23 Nov)

We have **7 weeks** until the demo on **Monday 23 November**: one preparation week (Week 0) and six working weeks. Every week runs **Monday → Sunday**. Your deadline is **Sunday 9:00 pm**: by then the week's tasks must be finished, tested and sent to GitHub as a **Pull Request** (section 5).

Week	By Sunday 9 pm you must finish...	Pull Request title
Week 0 Mon 5 – Sun 11 Oct	Study section 4. Install VS 2022 + Irvine in <code>C:\Irvine</code> . Fork, clone and build by Fri 9 Oct . Practice PR: fill the Week 0 line of <code>docs/progress/alyan.md</code> .	Week 0 – Alyan – practice PR
Week 1 Mon 12 – Sun 18 Oct	B1 <code>InitSystem</code> , <code>PutCell</code> , <code>PutString</code> , <code>ClearBuffer</code> .	Week 1 – Alyan – B1
Week 2 Mon 19 – Sun 25 Oct	B2 <code>Render</code> , <code>InvalidateScreen</code> , <code>DrawBorders</code> , <code>BuildFrame</code> → M1 (sub moves, no flicker).	Week 2 – Alyan – B2
Week 3 Mon 26 Oct – Sun 1 Nov	B3 <code>PutNumber</code> + <code>DrawHUD</code> .	Week 3 – Alyan – B3
Week 4 Mon 2 – Sun 8 Nov	B4 <code>StartExplosion</code> , <code>UpdateExplosions</code> , <code>DrawExplosions</code> → M2 .	Week 4 – Alyan – B4
Week 5 Mon 9 – Sun 15 Nov	B5 <code>StartSonar</code> , <code>UpdateSonar</code> , hidden-mine drawing, cool-down on the status bar.	Week 5 – Alyan – B5
Week 6 Mon 16 – Sun 22 Nov	By Wed 18 Nov : B6 (optional) and fixes — after that only bug fixes . Run your tests; write <code>docs/report/alyan.md</code> by Sat 21 Nov .	Week 6 – Alyan – fixes + report

2.1 The weekly rhythm (same for all three members)

When	What to do
Monday	Sync with Anas's repo (get last week's merged code) and start a new branch for the week.
Mon – Fri	Do the week's step(s). Commit + push every time a small piece works (at least every 2 days).
Saturday	Test your step(s) with the "Test it" boxes. Fix bugs.
Sunday before 9:00 pm	Write 3 lines in <code>docs/progress/alyan.md</code> , push, and open the Pull Request .
Sunday 9:30 pm	15-minute group call: what is done, what is stuck, next week's steps.
Sunday night / Monday	Anas merges all Pull Requests and tells the group "Week N merged — sync now".

2.2 Group milestones

Milestone	Date	What must work (whole group)
M1	25 Oct	Program runs; menu appears; submarine moves on screen with no flicker; random numbers work
M2	8 Nov	Core game: torpedoes, falling charges, hits, explosions, score, lives, status bar

Milestone	Date	What must work (whole group)
M3	18 Nov	Everything: sonar + hidden mines, sounds, levels, pause, high-score file, game-over screen
M4	22 Nov	All tests passed on a lab PC; final report written; ready for the 23 Nov demo

Tips

- **Behind schedule?** Say it in the Sunday call — don't hide it. Push only code that **builds**: an unfinished procedure keeps its STUB line, so the game still runs.
- Your plan is a little **ahead** of the proposal's timeline on purpose — the extra days are your safety buffer.

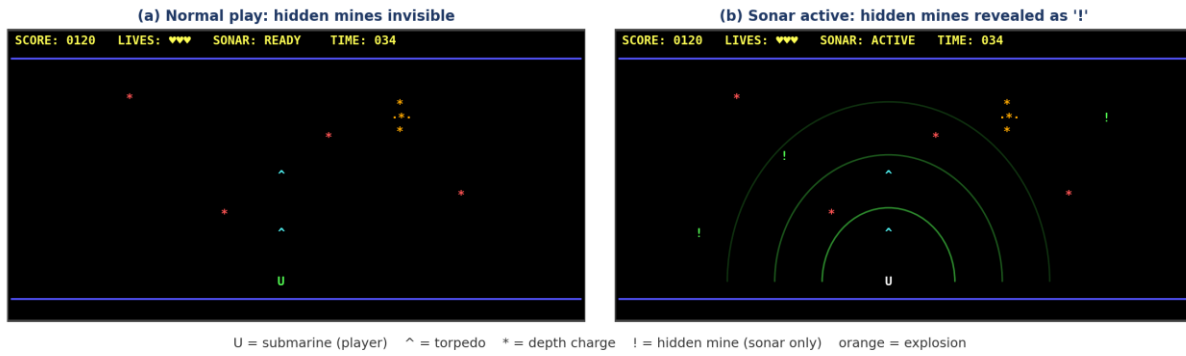
3. How the Game Works (5-minute version)

The whole game is **one loop** that repeats about **12 times per second** (every 80 ms). One repeat is called a **frame**. In every frame the program does the same five jobs, always in this order:

Job in every frame	What happens	Procedures	Who
1. Read keys	Check the keyboard without waiting ; move sub, fire, sonar, pause	<code>HandleInput</code>	Anas
2. Move things	Torpedoes go up, charges fall down, new charges appear, explosions age	<code>UpdateTorpedoes</code> <code>UpdateCharges</code> , <code>SpawnCharge</code> <code>UpdateExplosions</code>	Anas Ali Alyan (you)
3. Check hits	Torpedo hits charge → points; charge hits sub → lose a life	<code>CheckCollisions</code> → <code>AddScore</code> , <code>LoseLife</code>	Anas → Ali
4. Draw	Draw everything into a memory copy of the screen, then show only what changed	<code>BuildFrame</code> , <code>DrawHUD</code> , <code>Render</code>	Alyan (you)
5. Wait	Wait the rest of the 80 ms so the speed is the same on every PC	<code>FrameDelay</code>	Anas

Around the loop there are screens (menu, instructions, game over) and a **state** variable `gameState` that tells the loop what to do:

<code>gameState</code>	Meaning	What the loop does
ST_MENU (0)	Menu is showing	Loop stops, menu appears (ESC goes here)
ST_PLAYING (1)	Normal play	All five jobs
ST_SONAR (2)	Sonar is on (SPACE)	Nothing moves for 2 s; hidden mines shown as green 'I'
ST_PAUSED (3)	Paused (P)	Nothing moves; "PAUSED" text is drawn
ST_GAMEOVER (4)	No lives left	Loop stops, game-over screen appears
ST_EXIT (5)	Exit chosen in menu	Program ends



What the player sees: an 80×25 text screen (status bar on top, submarine 'U' at the bottom).

3.1 Project files and who owns them

There is **one** `.asm` file (`src/game.asm`). It **INCLUDEs** one `.inc` file per module. Every member has **one folder** and edits only that folder, so three people can work at the same time without conflicts. The full folder tree and the Git rules are in **section 5**.

Folder / file	What is inside	Owner
<code>src/game.asm</code>	<code>main</code> , <code>InitGame</code> , <code>GameLoop</code> , <code>FrameDelay</code> , all INCLUDE lines	Anas
<code>src/shared/constants.inc</code> , <code>data.inc</code>	Shared numbers and variables (everyone agrees first)	Anas (all agree)
<code>src/anas/</code>	<code>input.inc</code> , <code>torpedo.inc</code> , <code>collision.inc</code> , <code>stubs_anas.inc</code>	Anas
<code>src/alyan/</code>	<code>render.inc</code> , <code>hud.inc</code> , <code>explosion.inc</code> , <code>sonar.inc</code> , <code>stubs_alyan.inc</code>	Alyan ← you
<code>src/ali/</code>	<code>random.inc</code> , <code>screens.inc</code> , <code>charges.inc</code> , <code>sound.inc</code> , <code>score.inc</code> , <code>highscore.inc</code> , <code>stubs_ali.inc</code>	Ali
<code>docs/progress/<name>.md</code> <code>docs/report/<name>.md</code>	Your weekly log and your final-report section	Each member

3.2 The one object type everybody uses

The submarine is just a column number (`subX`). Every torpedo, charge and explosion is a 4-byte `GameObject` stored in an array. **active = 0** means the slot is free. **hidden = 1** means the charge is an invisible mine (it still moves and still hits — it is only *not drawn*).

```

data.inc (the comment shows who WRITES each variable; anyone may read it)
;=====
; data.inc - shared data (who WRITES each variable is in the comment)
;=====
GameObject STRUCT
    xPos    BYTE 0          ; column 0-79
    yPos    BYTE 0          ; row 0-24
    active   BYTE 0          ; 1 = slot in use
    hidden   BYTE 0          ; 1 = drawn only during sonar
GameObject ENDS

.data
gameState   BYTE  ST_MENU      ; Anas
subX        BYTE  40           ; Anas
score       DWORD 0            ; Ali
highScore   DWORD 0            ; Ali
lives       BYTE  3            ; Ali
level       BYTE  1            ; Ali
fireCooldown BYTE 0            ; Anas
lastFrame   DWORD 0            ; Anas
frameCount  DWORD 0            ; Anas
fallDelay   BYTE  6            ; Ali (set per level)
spawnDelay  BYTE  14           ; Ali (set per level)
fallTimer   BYTE  6            ; Ali
spawnTimer  BYTE  14           ; Ali
sonarStart  DWORD 0            ; Alyan
sonarReadyAt DWORD 0           ; Alyan
sonarPlayed BYTE  0            ; Alyan

Torpedoes   GameObject MAX_TORP DUP(<>) ; Anas
Charges     GameObject MAX_CHARGE DUP(<>) ; Ali
Explosions  GameObject MAX_EXPL  DUP(<>) ; Alyan
ExplTimer   BYTE  MAX_EXPL  DUP(0)      ; Alyan

ScreenBuf   WORD  SCREEN_W*SCREEN_H DUP(0720h) ; Alyan
PrevBuf     WORD  SCREEN_W*SCREEN_H DUP(0FFFFh) ; Alyan

```

4. What You Need to Know First

Study these in **Week 0 (5 – 11 Oct)** and keep going in Week 1 (Irvine, *Assembly Language for x86 Processors*, 8th ed.).

Topic	Simple explanation	Irvine	Practice
Basics, <code>MOV</code> , <code>MOVZX</code> , <code>CMP</code> + unsigned jumps, procedures with <code>USES</code>	Same basics as everyone	Ch 3–6	Write a procedure that adds two numbers
2-D array in a 1-D array	Cell (row, col) is element row × 80 + col . Each cell is a WORD (2 bytes), so the byte offset is ×2	Ch 9	Store and print an 8 × 4 grid of letters
<code>IMUL</code> , <code>DIV</code>	<code>imul ebx, ebx, 80</code> multiplies. <code>DIV</code> gives quotient in EAX and remainder in EDX (EDX = 0 first)	Ch 7	Turn index 165 back into row 2, col 5
String instructions <code>REP STOSW</code> , <code>REP MOVSD</code> , <code>CLD</code>	Fill or copy a whole array with one instruction (ECX = count, EDI = destination, ESI = source)	Ch 9	Fill an array with <code>REP STOSW</code> , copy it with <code>REP MOVSD</code>
<code>Gotoxy</code> , <code>SetTextColor</code> , <code>WriteChar</code> , <code>WriteString</code>	Move the cursor (DH = row, DL = col), choose a colour, print	Ch 5	Print a coloured '*' at row 10, col 20

Topic	Simple explanation	Irvine	Practice
Number → text	Divide by 10 again and again; each remainder + '0' is one digit	Ch 7	Print 345 digit by digit without <code>WriteDec</code>
<code>INVOKE</code> , <code>ADDR</code> , Win32 functions	Call Windows functions such as <code>SetConsoleCursorInfo</code>	Ch 11	Hide the console cursor
Lookup tables, parallel arrays	Use a number as an index into a table: <code>mov al, ExplChars[ecx]</code>	Ch 4	Print the day name for 0–6

5. GitHub: Folder Structure and How to Push Your Work

5.1 The folder structure (already created by Anas)

Repository: <https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection>. Every member has **one folder** in `src/`. Because nobody edits another person's files, Git can always merge the three parts **without conflicts**.

```

folder structure
Submarine-Defense-Game-with-Sonar-Detection/
├── README.md      .gitignore  .gitattributes  .github/CODEOWNERS      Anas
├── Project.sln    Project.vcxproj  Project.vcxproj.filters  Anas (others NEVER commit)
├── src/
│   ├── game.asm          Anas    the only file that is assembled
│   ├── shared/
│   │   ├── constants.inc  Anas    (all three agree first)
│   │   └── data.inc        Anas    (all three agree first)
│   ├── anas/
│   │   ├── input.inc      torpedo.inc  collision.inc
│   │   └── stubs anas.inc
│   ├── alyan/
│   │   ├── render.inc     hud.inc      explosion.inc  sonar.inc
│   │   └── stubs_alyan.inc
│   ├── ali/
│   │   ├── random.inc     screens.inc  charges.inc
│   │   ├── sound.inc      score.inc     highscore.inc
│   │   └── stubs ali.inc
│   └── docs/
│       ├── proposal/      guides/          Anas
│       ├── progress/      anas.md    alyan.md    ali.md      each member: own weekly log
│       └── report/        anas.md    alyan.md    ali.md      each member: own report section

```

	Files
You may change	<code>src/alyan/</code> (all files inside), <code>docs/progress/alyan.md</code> , <code>docs/report/alyan.md</code>
Never change	<code>src/game.asm</code> , <code>src/shared/</code> , <code>src/anas/</code> , <code>src/ali/</code> , <code>Project.*</code> , <code>README.md</code> , other members' docs

Rules that prevent conflicts

- Never use `git add .` — add only your own paths (shown in the commands below).
- Your module's private variables and tables go in a `.data` section **inside your own .inc file** (e.g. `FallDelayTable` lives in `score.inc`). Only truly shared variables are in `data.inc`.
- Need a new shared constant/variable, or a change in `game.asm`? Open a GitHub **Issue** or message Anas. He adds it.
- Never add files to the Visual Studio project — it changes `Project.vcxproj`. All files already exist.
- Testing something in `game.asm` on your PC is fine — run `git restore src/game.asm` before you commit.

- Everyone installs Irvine in **C:\Irvine**, so the same project settings work on all three laptops.

5.2 One-time setup (Week 0 — finish by Fri 9 Oct)

1. Install **Visual Studio 2022** (*Desktop development with C++*), Irvine32 in **C:\Irvine**, and **Git for Windows** (git-scm.com). Use the *Git Bash* window for the commands below.
2. Accept the **collaborator invitation** e-mail from GitHub (from muhammadanas20).
3. Open <https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection> and click **Fork** → **Create fork**. You now have your own copy on GitHub.
4. Run these commands (replace YOUR-USERNAME with your GitHub username):

```
Git Bash — one time
git config --global user.name "Syed Alyan Hussain"
git config --global user.email "the-email-of-your-github-account"

cd ~/source/repos
git clone https://github.com/YOUR-USERNAME/Submarine-Defense-Game-with-Sonar-Detection.git
cd Submarine-Defense-Game-with-Sonar-Detection

# connect Anas's repo as "upstream" (only once)
git remote add upstream https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection.git
git remote -v          # origin = your fork,  upstream = Anas's repo
```

5. Open **Project.sln** in Visual Studio. Build with **Ctrl+Shift+B**, run with **Ctrl+F5**. It must print *"Depth Charge skeleton builds and runs OK"*.
6. **Practice Pull Request:** do the weekly steps below once with only `docs/progress/alyan.md` (write your Week 0 line). When Anas merges it, you know the whole process works.

5.3 Every week, step by step

```
Git Bash — weekly
# ----- MONDAY: get last week's merged work, start this week's branch
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
git checkout -b week1-alyan          # week2-alyan, week3-alyan, ...

# ----- EVERY TIME SOMETHING SMALL WORKS
git status                          # must list ONLY src/alyan/... and docs/.../alyan.md
git add src/alyan docs/progress/alyan.md docs/report/alyan.md
git commit -m "B1: short description of what now works"
git push -u origin week1-alyan      # first push of the week (later: just  git push)

# ----- MID-WEEK (only if you need code Anas merged after Monday)
git fetch upstream
git merge upstream/main
```

Good commit messages start with the step number, e.g. **B2: Render skips unchanged cells** — then everyone can see what each commit did.

5.4 Sunday: open the Pull Request (before 9:00 pm)

1. Write your 3 lines in `docs/progress/alyan.md`, commit and push (commands above).
2. Open **your fork** on GitHub. Click the yellow **Compare & pull request** button (or: *Pull requests* → *New pull request*).

3. Check the arrows: **base repository: muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection, base: main ← head repository: YOUR-USERNAME/Submarine-Defense-Game-with-Sonar-Detection, compare: week1-alyan.**
4. Title: the one from your timeline, e.g. "Week 1 – Alyan – B1". Description: what works, how you tested it, any problems.
5. Open the **Files changed** tab: only `src/alyan/` and your own docs files may appear. If something else appears, fix it first (table below).
6. Click **Create pull request**. Anas merges it on Sunday night; on Monday you sync again (5.3).

5.5 If something goes wrong

Problem	Fix
<code>git status</code> lists <code>src/game.asm</code> (you tested something)	<code>git restore src/game.asm</code>
<code>Project.vcxproj / .sln / .filters</code> changed	<code>git restore Project.sln Project.vcxproj Project.vcxproj.filters</code>
You already committed a file outside your folder	<code>git checkout upstream/main -- <file></code> then commit "Undo change" and push
Conflict while merging <code>upstream/main</code>	Can only happen in your own files. Open the file, keep the right lines, delete the <code><<<<<<, =====, >>>>>></code> lines, then <code>git add <file></code> and <code>git commit</code> . Unsure? Ask Anas before pushing.
Push rejected ("fetch first")	<code>git pull origin <your-branch></code> then <code>git push</code>
Build error A2005 <i>symbol redefinition</i>	You wrote a procedure but its STUB line is still there — delete the STUB line.
Build error A2006 <i>undefined symbol</i>	The procedure has no real version and no STUB line — tell its owner / Anas.
Which branch am I on?	<code>git branch</code> (the one with *)

5.6 Your stub file

`src/alyan/stubs_alyan.inc` lets the game build before your code exists. When you finish a procedure, delete its **STUB** line **in the same commit**.

```
src/alyan/stubs_alyan.inc (starting version)
; stubs_alyan.inc - EMPTY versions of Alyan's procedures.
; Alyan deletes ONE line as soon as the real procedure is written.
STUB MACRO procName          ; makes:  procName PROC / ret / procName ENDP
procName PROC
    ret
procName ENDP
ENDM

.code
STUB InitSystem              ; called by main
STUB InvalidateScreen        ; called by InitGame
STUB BuildFrame              ; called by GameLoop
STUB Render                  ; called by GameLoop
STUB UpdateExplosions        ; called by GameLoop
STUB StartExplosion           ; called by CheckCollisions
STUB StartSonar              ; called by HandleInput
STUB UpdateSonar             ; called by GameLoop
STUB DrawHUD                 ; called by BuildFrame
STUB DrawExplosions          ; called by BuildFrame
```

Each `STUB` line creates an empty procedure that just returns. (`STUB` is a small **macro** — a good example of Irvine Ch 10.)

6. Your Steps

Do the steps in order. Each step says **which file**, **what it does in simple words**, the **logic**, an example, how to **test** it and the usual **mistakes**.

B1 Console setup and the screen buffer (Week 1 · 12 – 18 Oct)

File	src/alyan/render.inc
You write	<code>InitSystem</code> , <code>PutCell</code> , <code>PutString</code> , <code>ClearBuffer</code>
You call	<code>SeedRandom</code> (Ali), <code>SetConsoleTitle</code> , <code>SetConsoleCursorInfo</code>
Called by	main, and all your drawing code

In simple words

- `ScreenBuf` is 2000 WORDs = 80×25 cells. In each WORD: **low byte** = **character**, **high byte** = **colour**.
- `PutCell` = "put this character with this colour at row DH, column DL" — in memory only, not on screen yet.

The logic, step by step

1. `InitSystem`: call `SeedRandom`, set the window title, hide the blinking cursor.
2. `PutCell`: `index = row × 80 + col` (`movzx + imul`), then `mov ScreenBuf[ebx*2], ax`.
3. `PutString`: loop over a 0-terminated string (ESI) and call `PutCell` for each character, moving one column right.
4. `ClearBuffer`: fill all 2000 WORDs with 0720h (space, light grey) using `REP STOSW`.

render.inc – InitSystem

```

;-----
; InitSystem: one-time console setup
;-----
InitSystem PROC USES eax
    call SeedRandom
    INVOKE SetConsoleTitle, ADDR titleStr
    INVOKE GetStdHandle, STD_OUTPUT_HANDLE
    INVOKE SetConsoleCursorInfo, eax, ADDR cursorInfo
    ret
InitSystem ENDP

```

render.inc – PutCell

```

;-----
; PutCell: ScreenBuf[row*80 + col] = AX
; Receives: DH = row, DL = col, AL = char, AH = colour
;-----
PutCell PROC USES ebx ecx
    movzx ebx, dh
    imul ebx, ebx, SCREEN_W    ; ebx = row * 80
    movzx ecx, dl
    add ebx, ecx               ; ebx = row*80 + col (cell index)
    mov ScreenBuf[ebx*2], ax   ; *2 because each cell is a WORD
    ret
PutCell ENDP

```

render.inc – PutString, ClearBuffer

```

;-----
; PutString: ESI = offset of 0-terminated string,
;           DH = row, DL = col, AH = colour
;-----
PutString PROC USES eax edx esi

```

```

NextChar:
    mov     al, [esi]
    cmp     al, 0
    je      Done
    call    PutCell
    inc     dl
    inc     esi
    jmp     NextChar
Done:
    ret
PutString ENDP

;-----
; ClearBuffer: fills ScreenBuf with blanks using REP STOSW
;-----
ClearBuffer PROC USES eax ecx edi
    cld
    mov     ax, (ATTR_TEXT SHL 8) OR ' '    ; 0720h
    mov     edi, OFFSET ScreenBuf
    mov     ecx, SCREEN_W * SCREEN_H
    rep     stosw
    ret
ClearBuffer ENDP

```

Test it — the step is DONE when...

- In the Memory window at `ScreenBuf: PutCell` with row 1, col 2 changes the WORD at byte offset $(1 \times 80 + 2) \times 2 = 164$.
- The blinking cursor is gone.

Common mistakes

- Forgetting $\times 2$ (`[ebx*2]`) — characters land in the wrong place.
- Using DH directly as a 32-bit index — `movzx ebx, dh` first.

B2 Renderer and frame builder (Week 2 · 19 – 25 Oct)

File	src/alyan/render.inc
You write	<code>Render</code> , <code>InvalidateScreen</code> , <code>DrawBorders</code> , <code>BuildFrame</code>
You call	<code>Gotoxy</code> , <code>SetTextColor</code> , <code>WriteChar</code> , <code>DrawHUD</code> (B3), <code>DrawExplosions</code> (B4)
Called by	<code>GameLoop</code> (every frame), <code>InitGame</code>

In simple words

- `PrevBuf` remembers what is **already** on the screen.
- `Render` walks through all 2000 cells. If a cell is the same in both buffers, skip it. If it is different, draw it.
- `BuildFrame` paints the new picture: clear → status bar → borders → sub → torpedoes → charges → explosions.

The logic, step by step

1. `Render`: for $i = 0 \dots 1998$, compare `ScreenBuf[i]` with `PrevBuf[i]`. If different: row = $i / 80$, col = $i \bmod 80$ (one `DIV`), then `Gotoxy`, `SetTextColor`, `WriteChar`.
2. Stop one cell early: writing the very last cell (row 24, col 79) makes the console scroll.
3. Afterwards copy `ScreenBuf` into `PrevBuf` with `REP MOVSD` (2000 WORDs = 1000 DWORDs).
4. `InvalidateScreen`: fill `PrevBuf` with `0FFFFh`, so every cell looks "changed" and the next `Render` redraws all.

5. **BuildFrame**: draw each object with **PutCell**. For charges: if **hidden = 1**, draw it **only** in **ST_SONAR** (as a green '!'); otherwise skip it — it still exists in memory.

```
render.inc - Render
;-----
; Render: draws only the cells that changed since the last frame
;-----
Render PROC USES eax ebx ecx edx esi edi
    xor ebx, ebx                ; ebx = cell index 0..1999
NextCell:
    cmp ebx, SCREEN_W * SCREEN_H - 1 ; never write the bottom-right
    jae CopyFrame              ; cell: it would scroll the console
    mov ax, ScreenBuf[ebx*2]
    cmp ax, PrevBuf[ebx*2]
    je Skip                    ; unchanged -> no console call
    push eax                   ; save char + colour
    mov eax, ebx
    xor edx, edx
    mov ecx, SCREEN_W
    div ecx                    ; EAX = row, EDX = col
    mov dh, al                 ; DH = row, DL = col
    call Gotoxy
    mov eax, [esp]             ; peek saved char/colour
    movzx eax, ah
    call SetTextColor          ; EAX = colour
    pop eax
    call WriteChar             ; AL = char
Skip:
    inc ebx
    jmp NextCell
CopyFrame:
    cld                        ; PrevBuf = ScreenBuf
    mov esi, OFFSET ScreenBuf
    mov edi, OFFSET PrevBuf
    mov ecx, (SCREEN_W * SCREEN_H) / 2 ; 2 WORDs per DWORD
    rep movsd
    ret
Render ENDP
```

```
render.inc - InvalidateScreen, DrawBorders
;-----
; InvalidateScreen: fills PrevBuf with 0FFFFh so the next Render
; redraws every cell (call after Clrscr / menus)
;-----
InvalidateScreen PROC USES eax ecx edi
    cld
    mov ax, 0FFFFh
    mov edi, OFFSET PrevBuf
    mov ecx, SCREEN_W * SCREEN_H
    rep stosw
    ret
InvalidateScreen ENDP

;-----
; DrawBorders: '=' lines on row 1 and row 24 (cols 0..78)
;-----
DrawBorders PROC USES eax edx
    mov al, CH_BORDER
    mov ah, ATTR_BORDER
    mov dl, 0
B1: mov dh, 1
    call PutCell
    mov dh, BOTTOM_ROW
    call PutCell
    inc dl
    cmp dl, SCREEN_W - 1 ; skip col 79 (avoids scrolling)
```

```

    jb    B1
    ret
DrawBorders ENDP

```

render.inc - BuildFrame

```

;-----
; BuildFrame: draws every object into ScreenBuf
;-----
BuildFrame PROC USES eax ecx edx esi
    call ClearBuffer
    call DrawHUD
    call DrawBorders

    ; submarine
    mov dh, SUB_ROW
    mov dl, subX
    mov al, CH_SUB
    mov ah, ATTR_SUB
    call PutCell

    ; torpedoes
    mov esi, OFFSET Torpedoes
    mov ecx, MAX_TORP
T1: cmp (GameObject PTR [esi]).active, 0
    je TN
    mov dh, (GameObject PTR [esi]).yPos
    mov dl, (GameObject PTR [esi]).xPos
    mov al, CH_TORP
    mov ah, ATTR_TORP
    call PutCell
TN: add esi, TYPE GameObject
    loop T1

    ; charges and hidden mines
    mov esi, OFFSET Charges
    mov ecx, MAX_CHARGE
C1: cmp (GameObject PTR [esi]).active, 0
    je CN
    mov dh, (GameObject PTR [esi]).yPos
    mov dl, (GameObject PTR [esi]).xPos
    cmp (GameObject PTR [esi]).hidden, 0
    je Visible
    cmp gameState, ST_SONAR
    jne CN ; hidden + no sonar: NOT drawn
    mov al, CH_MINE ; (but it still exists in memory)
    mov ah, ATTR_MINE
    jmp DrawIt
Visible:
    mov al, CH_CHARGE
    mov ah, ATTR_CHARGE
DrawIt:
    call PutCell
CN: add esi, TYPE GameObject
    loop C1

    call DrawExplosions
    ret
BuildFrame ENDP

```

Until B3 and B4 are done, `DrawHUD` and `DrawExplosions` are still stubs in `stubs_alyan.inc`, so this already builds.

Test it — the step is DONE when...

- **Milestone M1 (25 Oct) with Anas and Ali:** the sub moves on screen with **no flicker**.
- A breakpoint in the `skip` branch shows that most cells are skipped every frame.

Common mistakes

- Calling `Clrscr` every frame — heavy flicker. Only clear the **buffer**.
- Writing the bottom-right cell — the screen scrolls up one line.
- Special characters (—, ♥) may print as '?'. Stick to normal ASCII like '='.

B3 Status bar (HUD) and number printing (Week 3 · 26 Oct – 1 Nov)

File	src/alyan/hud.inc
You write	<code>PutNumber</code> , <code>DrawHUD</code>
You call	<code>PutString</code> , <code>PutCell</code> , <code>GetMseconds</code>
Called by	<code>BuildFrame</code>

In simple words

- Row 0 shows: **SCORE 00035 LIVES 3 LEVEL 1 SONAR READY**.
- To show a number we must turn it into digit characters ourselves: divide by 10, the remainder is the last digit.

The logic, step by step

1. `PutNumber`: 5 times: `xor edx, edx` → `div` by 10 → remainder + '0' → store from right to left in `numBuf`. Then `PutString`.
2. `DrawHUD`: write the labels with `PutString`; score with `PutNumber`; lives and level are one digit (+ '0').
3. Sonar part: in `ST_SONAR` show ACTIVE; else if now $\geq$ `sonarReadyAt` show READY; else show the seconds left.
4. If `gameState = ST_PAUSED`, also write "PAUSED" in the middle of the screen.

```

hud.inc - PutNumber
;-----
; PutNumber: writes EAX as 5 decimal digits (leading zeros)
; Receives: EAX = value, DH = row, DL = col, BL = colour
;-----
PutNumber PROC USES eax ebx ecx edx esi edi
    push edx                ; save row/col (DIV uses EDX)
    mov edi, OFFSET numBuf + 4 ; fill from the right
    mov esi, 5
    mov ecx, 10
DigitLoop:
    xor edx, edx
    div ecx                  ; EAX = EAX/10, EDX = digit
    add dl, '0'              ; digit -> ASCII
    mov [edi], dl
    dec edi
    dec esi
    jnz DigitLoop
    pop edx
    mov esi, OFFSET numBuf
    mov ah, bl
    call PutString
    ret
PutNumber ENDP

```

hud.inc - DrawHUD

```

;-----
; DrawHUD: SCORE, LIVES, LEVEL, SONAR status on row 0
;-----
DrawHUD PROC USES eax ebx edx esi
    mov  ah, ATTR HUD
    mov  bl, ATTR_HUD
    mov  dh, HUD_ROW

    mov  dl, 1
    mov  esi, OFFSET strScore
    call PutString
    mov  dl, 8
    mov  eax, score
    call PutNumber

    mov  ah, ATTR HUD
    mov  dl, 17
    mov  esi, OFFSET strLives
    call PutString
    mov  dl, 24
    mov  al, lives
    add  al, '0'
    call PutCell

    mov  dl, 29
    mov  esi, OFFSET strLevel
    call PutString
    mov  dl, 36
    mov  al, level
    add  al, '0'
    call PutCell

    mov  dl, 41
    mov  esi, OFFSET strSonar
    call PutString
    mov  dl, 48
    cmp  gameState, ST_SONAR
    je   SonarOn
    call GetMseconds
    cmp  eax, sonarReadyAt
    jae  SonarReady
    ; cooling down: show seconds left = (readyAt - now) / 1000 + 1
    mov  ebx, sonarReadyAt
    sub  ebx, eax
    mov  eax, ebx
    xor  edx, edx
    mov  ebx, 1000
    div  ebx
    inc  eax
    mov  dh, HUD ROW
    mov  dl, 48
    mov  bl, ATTR_HUD
    call PutNumber
    jmp  PauseText
SonarOn:
    mov  esi, OFFSET strActive
    call PutString
    jmp  PauseText
SonarReady:
    mov  ah, ATTR_HUD                ; GetMseconds destroyed AH
    mov  esi, OFFSET strReady
    call PutString
PauseText:
    cmp  gameState, ST_PAUSED
    jne  Done
    mov  dh, 12
    mov  dl, 28
    mov  ah, ATTR_HUD
    mov  esi, OFFSET strPaused

```

```

    call PutString
Done:
    ret
DrawHUD ENDP

```

Test it — the step is DONE when...

- The score shows 00000, 00010, 00035 ...; lives and level change at once.

Common mistakes

- Keeping row/col in EDX while dividing — `DIV` overwrites EDX. `push edx` first.
- `GetMseconds` overwrites EAX, so the colour in AH must be set again afterwards.

B4 Explosions (Week 4 · 2 – 8 Nov)

File	src/alyan/explosion.inc
You write	<code>StartExplosion</code> , <code>UpdateExplosions</code> , <code>DrawExplosions</code>
You call	<code>PutCell</code>
Called by	<code>CheckCollisions</code> (Anas), <code>GameLoop</code> , <code>BuildFrame</code>

In simple words

- An explosion lives for 5 frames. Each frame it shows a different character and colour: . + * # @
- A small timer array (`ExplTimer`) counts down; at 0 the explosion slot is free again.

The logic, step by step

1. `StartExplosion` (AL = col, AH = row): find a free slot, store the position, set its timer to `EXPL_FRAMES`.
2. `UpdateExplosions`: for each active slot, `dec` the timer; at 0 set active = 0.
3. `DrawExplosions`: index = timer – 1; take the character and colour from two lookup tables; `PutCell`.

```

explosion.inc
;-----
; StartExplosion: AL = col, AH = row
;-----
StartExplosion PROC USES ebx esi
    mov esi, OFFSET Explosions
    xor ebx, ebx                ; ebx = index
Find:
    cmp ebx, MAX_EXPL
    jae Done                    ; no free slot: skip effect
    cmp (GameObject PTR [esi + ebx*TYPE GameObject]).active, 0
    je Found
    inc ebx
    jmp Find
Found:
    mov (GameObject PTR [esi + ebx*TYPE GameObject]).xPos, al
    mov (GameObject PTR [esi + ebx*TYPE GameObject]).yPos, ah
    mov (GameObject PTR [esi + ebx*TYPE GameObject]).active, 1
    mov ExplTimer[ebx], EXPL_FRAMES
Done:
    ret
StartExplosion ENDP

UpdateExplosions PROC USES ebx esi
    mov esi, OFFSET Explosions

```

```

    xor     ebx, ebx
L1:  cmp     (GameObject PTR [esi + ebx*TYPE GameObject]).active, 0
    je      NextE
    dec     ExplTimer[ebx]
    jnz     NextE
    mov     (GameObject PTR [esi + ebx*TYPE GameObject]).active, 0
NextE:
    inc     ebx
    cmp     ebx, MAX_EXPL
    jb      L1
    ret
UpdateExplosions ENDP

DrawExplosions PROC USES eax ebx ecx edx esi
    mov     esi, OFFSET Explosions
    xor     ebx, ebx
L1:  cmp     (GameObject PTR [esi + ebx*TYPE GameObject]).active, 0
    je      NextE
    movzx   ecx, ExplTimer[ebx]          ; 1..5
    dec     ecx                          ; table index 0..4
    mov     al, ExplChars[ecx]
    mov     ah, ExplColors[ecx]
    mov     dh, (GameObject PTR [esi + ebx*TYPE GameObject]).yPos
    mov     dl, (GameObject PTR [esi + ebx*TYPE GameObject]).xPos
    call    PutCell
NextE:
    inc     ebx
    cmp     ebx, MAX_EXPL
    jb      L1
    ret
DrawExplosions ENDP

```

Test it — the step is DONE when...

- Every hit shows a short animated explosion at the right place.
- **Milestone M2 (8 Nov):** core game playable with the whole group.

Common mistakes

- Timer index off by one (table index must be timer – 1).
- Forgetting `TYPE GameObject` scaling in `[esi + ebx*TYPE GameObject]`.

B5 Sonar mode and hidden mines (Week 5 · 9 – 15 Nov)

File	src/alyan/sonar.inc (+ the hidden-mine part of BuildFrame and DrawHUD)
You write	<code>StartSonar</code> , <code>UpdateSonar</code>
You call	<code>GetMseconds</code> , <code>BuildFrame</code> , <code>Render</code> , <code>PlaySonarSweep</code> (Ali)
Called by	<code>HandleInput</code> (SPACE), <code>GameLoop</code>

In simple words

- SPACE → the game **freezes for 2 seconds** and hidden mines appear as green '!'. Then they vanish again.
- After that the sonar needs **10 seconds** to recharge (the status bar counts down).
- Key idea for the viva: a mine can be *invisible on the screen* but still *real in memory*.

The logic, step by step

1. `StartSonar`: `gameState = ST_SONAR`, remember the start time, `sonarPlayed = 0`.
2. While in `ST_SONAR`, `GameLoop` calls only `UpdateSonar` + drawing, so nothing moves.
3. `UpdateSonar`, first frame: draw the frame (so the '!' marks show) and play the sweep sound once.

4. When 2000 ms have passed: back to `ST_PLAYING` and set `sonarReadyAt = now + 10 000`.

```
sonar.inc
StartSonar PROC USES eax
    mov     gameState, ST_SONAR      ; freezes updates in GameLoop
    call    GetMseconds
    mov     sonarStart, eax
    mov     sonarPlayed, 0
    ret
StartSonar ENDP

UpdateSonar PROC USES eax
    cmp     sonarPlayed, 0
    jne     CheckTime
    call    BuildFrame                ; show the '!' marks first
    call    Render
    call    PlaySonarSweep            ; ~250 ms, game is frozen anyway
    mov     sonarPlayed, 1
CheckTime:
    call    GetMseconds
    sub     eax, sonarStart            ; elapsed = now - start
    cmp     eax, SONAR_MS
    jb      StillOn
    mov     gameState, ST_PLAYING     ; back to normal play
    call    GetMseconds
    add     eax, SONAR_COOLDOWN_MS
    mov     sonarReadyAt, eax
StillOn:
    ret
UpdateSonar ENDP
```

Test it — the step is DONE when...

- SPACE freezes the game for 2 s; hidden mines show as '!' and disappear again.
- SPACE during the cool-down does nothing; the status bar counts 10 ... 1, then READY.

B6 Optional speed-up and polish (Week 6 · 16 – 18 Nov)

Only if everything else works. `RenderFast` uses the string instruction `REPE CMPSW` to jump over many unchanged cells in one instruction, and updates `PrevBuf` while drawing (so no copy is needed). It is a nice extra point in the viva, but `Render` from B2 is enough.

```
render.inc - RenderFast (optional)
;-----
; RenderFast (optional upgrade): same result as Render, but uses
; REPE CMPSW to jump over unchanged cells in one instruction.
;-----
RenderFast PROC USES eax ebx ecx edx esi edi
    cld
    mov     esi, OFFSET ScreenBuf
    mov     edi, OFFSET PrevBuf
    mov     ecx, SCREEN_W * SCREEN_H - 1 ; skip the bottom-right cell
Scan:
    repe    cmpsw                        ; compare until a difference or ECX = 0
    je      Done                        ; ZF = 1 -> last pair equal: no more changes
    mov     ax, [esi-2]                  ; the changed cell (ESI already moved on)
    mov     [edi-2], ax                  ; PrevBuf = ScreenBuf for this cell
    push    eax
    mov     eax, esi
    sub     eax, OFFSET ScreenBuf + 2
    shr     eax, 1                      ; byte offset / 2 = cell index
    xor     edx, edx
    mov     ebx, SCREEN_W
    div     ebx                          ; EAX = row, EDX = col
    jmp     Scan
```

```

    mov  dh, al
    call Gotoxy
    mov  eax, [esp]
    movzx eax, ah
    call SetTextColor
    pop  eax
    call WriteChar
    test ecx, ecx          ; more cells left to scan?
    jnz  Scan
Done:
    ret
RenderFast ENDP

```

7. Viva Preparation

Questions about your part

1. Where is row 5, column 10 in `ScreenBuf`? ($(5 \times 80 + 10) \times 2 = 820$ bytes)
2. Why two buffers? What happens if the whole screen is redrawn every frame?
3. What do `REP STOSW` and `REP MOVSD` do? Which registers do they use?
4. How do you turn a number into digits on the status bar?
5. How can a mine be invisible but still destroy the submarine?
6. Why does `Render` stop one cell before the end of the screen?
7. How does the explosion animation pick its character and colour?

Questions about the other members' parts (you can be asked these too)

1. How does `HandleInput` read keys without waiting? — Anas
2. Why two different counters in `CheckCollisions`? — Anas
3. How does `Rand` give a number between 0 and 77? — Ali
4. Why does the sound macro use `PUSHAD/POPAD`? — Ali

8. Your Checklist

✓	Step	Target date
☐	Forked, cloned, built; practice PR merged	Fri 9 – Sun 11 Oct
☐	Studied the topics in section 4	Sun 11 Oct
☐	B1 InitSystem, PutCell, PutString, ClearBuffer — PR sent	Sun 18 Oct
☐	B2 Render, InvalidateScreen, borders, BuildFrame — PR sent (M1)	Sun 25 Oct
☐	B3 PutNumber + DrawHUD — PR sent	Sun 1 Nov
☐	B4 explosions — PR sent (M2)	Sun 8 Nov
☐	B5 sonar, '!' marks, cool-down on status bar — PR sent	Sun 15 Nov
☐	B6 (optional) RenderFast; all your STUB lines deleted (M3)	Wed 18 Nov
☐	Your tests passed; docs/report/alyan.md written	Sat 21 Nov

About the example code

All code comes from one reference version of the game (folder `guide/reference_code/`). It was checked with a MASM-compatible assembler (JWasm) and the Irvine32 include files — the full game, the day-one starter repo and the in-between versions with stub files all assemble with 0 errors — but it has **not** been run on Windows yet. Test every step on your own PC exactly as the "Test it" boxes say, and treat the code as a starting point that you must understand.